ELP: Empirical Lane Parallelism

Un método para ingeniería de software con múltiples agentes en paralelo, orquestados por humanos

Eduardo Díaz Cortés

2026-05-13

Contents

ELP: Empirical Lane Parallelism . . . 1
Resumen . . . 2
1. Motivación . . . 2
2. Definiciones . . . 2
3. Punto de partida del proyecto . . . 3
4. Roles . . . 4
5. Anatomía de una lane . . . 6
6. El brief como contrato . . . 6
7. Captura empírica . . . 8
8. Patrones operativos . . . 9
9. Reglas absolutas . . . 10
10. Anti-patrones . . . 11
11. Cuándo aplicar y cuándo no . . . 11
12. Comparación con métodos cercanos . . . 11
13. Métricas . . . 12
14. Caso de estudio: compilador para un lenguaje funcional con efectos algebraicos . . . 13
15. Limitaciones conocidas . . . 15
16. Origen y referencias . . . 16
Apéndice A. Plantilla mínima de brief . . . 16
Apéndice B. Plantilla mínima de retro de lane . . . 17

ELP: Empirical Lane Parallelism

Un método para construir software con múltiples agentes de IA trabajando en paralelo, bajo orquestación humana, con captura empírica del proceso.

"Welcome back, my friends, to the show that never ends."

Emerson, Lake & Palmer, *Karn Evil 9: First Impression Part 2*, 1973.

Resumen

ELP (*Empirical Lane Parallelism*) es un método de ingeniería de software que combina tres ideas:

1. **Lanes paralelas.** Cada unidad de trabajo se ejecuta como una sesión aislada: un *git worktree*, una rama propia, una sesión `tmux` propia. Varias lanes corren en paralelo.
2. **Brief como contrato.** Cada lane recibe un documento estructurado que define su alcance, las decisiones ya tomadas, qué hacer y qué no, y qué entregar.
3. **Captura empírica del proceso.** Auditorías previas al código, retrospectivas obligatorias por lane y mediciones de cada compilación se conservan como conjunto de datos que alimenta a las lanes siguientes.

ELP no reemplaza a los métodos clásicos de desarrollo, los complementa con un patrón explícito para coordinar a varios agentes sin que el sistema se degrade en pocas semanas.

El método se probó empíricamente en proyectos *greenfield*, es decir, proyectos nuevos, sin código heredado, que parten con visión de producto y diseño general definidos pero sin implementación previa. ELP se extiende de forma natural a proyectos con visión de producto y un *backlog* bien definido, aunque la práctica sostenida fuera del caso *greenfield* queda como trabajo futuro.

---

1. Motivación

Trabajar con asistentes de IA en una sola conversación funciona bien para tareas pequeñas. A medida que el proyecto crece, el patrón se rompe:

- El **contexto se llena**. El agente pierde detalles relevantes que estaban al inicio.
- Los **cambios se pisan** entre sí cuando varias tareas se intercalan.
- **No queda registro** de lo que se intentó y por qué se descartó.
- **Cada conversación parte de cero**, sin acceso al aprendizaje previo.

Los métodos del espectro actual (*vibe coding*, *Pantser*, SDD, SPDD, BMAD) no ofrecen una respuesta operativa al problema cuando se combinan con escala (decenas de PRs por día) y con duración (proyectos de varios meses). ELP apunta a llenar ese vacío.

---

2. Definiciones

Lane. Unidad de trabajo paralelizable, asociada a un único item del *backlog* (típicamente una tarjeta de un tablero Kanban o un *issue* de GitHub). Cada lane se materializa en un *git worktree* con su rama, su brief y su sesión `tmux`. El agente que ejecuta una lane corre en **modo autónomo** (en Claude, esto corresponde a *auto mode on*): ejecuta el brief de principio a fin sin pausas para pedir confirmación, salvo en los puntos de *STOP-and-report* declarados explícitamente.

Brief. Documento estructurado que define el contrato de la lane. Es la entrada del agente que ejecuta la lane. El nombre viene de *briefing*: antes del lanzamiento, el arquitecto y el integrador realizan un *briefing*, una puesta en común explícita, y el documento resultante es lo que recibe el agente como contrato. El brief **complementa** al item del *backlog* (que captura el *qué* funcional) con instrucciones tácticas (el *cómo* operativo para esa ejecución concreta).

Kanban. Tablero (físico o digital) donde el arquitecto gestiona las iniciativas, hace *triage* de prioridades y decide qué se aborda en la próxima lane. ELP no prescribe la herramienta específica. En

la práctica inicial se usaron *issues* de GitHub como sustituto funcional. Lo importante es que exista un lugar único y consultable donde el *qué construir* viva separado del *cómo construir* (que vive en el brief).

Integrador. Agente de IA con sesión interactiva que mantiene el contexto compartido con el humano, lee el estado del repositorio, escribe los briefs, lanza las lanes y monitorea su progreso. No toca código directamente.

Arquitecto. El humano. Decide el alcance, el orden de las lanes, integra los PRs, autoriza acciones irreversibles y corrige cuando el integrador o las lanes se equivocan.

Auditoría (*Phase 0*). Lane puramente documental que valida la premisa de un hito con datos antes de escribir código. Puede descartar la propuesta completa.

Lane retro. Documento que cada lane escribe **antes** de abrir el PR, con métricas, errores encontrados, ambigüedades de la especificación y puntos de fricción.

TSV de compilaciones. Archivo separado por tabuladores donde cada invocación a *build* o *test* queda registrada con marca de tiempo, comando, resultado y duración.

***STOP-and-report*.** Comportamiento del agente lane cuando encuentra una situación fuera de su alcance: se detiene y reporta, no improvisa.

---

3. Punto de partida del proyecto

ELP no parte del vacío. Asume condiciones iniciales explícitas, sin las cuales el método no se sostiene. El alcance probado empíricamente es **greenfield**. La extensión a proyectos con *backlog* preexistente es natural, pero exige las mismas precondiciones.

3.1 Documentación previa

El proyecto arranca con un cuerpo mínimo de documentación que el integrador puede leer y que sirve como fuente compartida para escribir los briefs. Típicamente incluye:

- **Visión de producto.** Qué se va a construir, para quién y qué problema resuelve.
- **Diseño general o arquitectura objetivo.** Componentes principales, lenguaje, plataforma de ejecución, decisiones tecnológicas.
- **Ejemplos y prototipos.** Fragmentos de la sintaxis objetivo, maquetas de interfaz, un *spike* sobre una pieza crítica, repositorios de referencia.
- **Glosario o modelo de dominio.** Términos del problema y cómo se relacionan entre sí.

La calidad de los briefs es función directa de esta documentación. Sin una visión escrita, el integrador inventa el alcance. Sin un diseño general, las lanes derivan en direcciones incompatibles.

3.2 Principios fijados

Antes de la primera lane se establecen, idealmente por escrito en el repositorio (`docs/principles.md` o equivalente):

- **Principios arquitectónicos.** Invariantes que ninguna lane puede romper. Por ejemplo, *"selfhost byte-identical en cada PR"*, *"sin dependencias de ejecución en X"*, *"toda función observable debe estar instrumentada"*.

- **Principios de negocio** (cuando aplican). Qué prioriza el producto en caso de compensación. Por ejemplo, *"latencia sobre rendimiento"*, *"compatibilidad heredada sobre velocidad de lanzamiento"*.
- **Restricciones de tiempo.** Metas y fechas de lanzamiento, hoja de ruta de versiones, ventanas de congelamiento e hitos externos comprometidos.

Los principios funcionan como axiomas del flujo. El brief los cita, no los discute de nuevo. Cambiar un principio es una decisión consciente del arquitecto, no algo que una lane pueda hacer al pasar.

3.3 Backlog gestionado en Kanban

El *qué construir* vive en un **tablero Kanban** que el arquitecto usa para gestionar las iniciativas y hacer el *triage* de qué se aborda a continuación. ELP no prescribe la herramienta concreta. Puede ser un tablero dedicado (Linear, Jira, Trello, GitHub Projects) o el conjunto de *issues* de GitHub como sustituto funcional (la implementación inicial de ELP usó este último).

Lo importante es que el tablero cumpla tres funciones:

1. **Inventario.** Cada item es una unidad candidata a convertirse en lane.
2. **Priorización.** El arquitecto ordena, agrupa y descarta items. El integrador lee ese orden.
3. **Trazabilidad.** Cada brief, retro y PR referencia al item del Kanban del que proviene.

El brief **complementa** al item del Kanban. El item captura el *qué* funcional, con el detalle que el arquitecto considere necesario, y el brief añade el *cómo* táctico (decisiones fijadas, archivos a tocar, alcance explícito) como resultado del *briefing* entre arquitecto, integrador y agente lane.

3.4 Greenfield versus proyectos con backlog preexistente

Dimensión	Greenfield (probado)	Con backlog preexistente (extensión natural)
Código de partida	ninguno o *spike* inicial	base existente con convenciones
Visión de producto	recién escrita	madura, posiblemente versionada
Principios arquitectónicos	se establecen al inicio	se documentan extrayéndolos del código
Tablero Kanban	nuevo, se va llenando	preexistente, con deuda y prioridades dadas
Riesgo principal	derivar sin guías	imponer ELP sobre un flujo que ya funciona

La extensión a proyectos que no son *greenfield* requiere un paso previo: **explicitar lo que está implícito** (principios, convenciones, contratos no escritos) para que los briefs los puedan citar.

---

4. Roles

ELP define tres roles con responsabilidades claras y no superpuestas.

4.1 Arquitecto (humano)

Responsabilidades:

- Decidir el alcance de cada hito y el orden de ejecución entre lanes.
- Aprobar o rechazar los PRs, o habilitar la integración automática bajo ciertas condiciones.
- Autorizar de forma explícita las acciones irreversibles (*force push*, limpieza destructiva, *push* a *main*).
- Corregir cuando el integrador o las lanes se equivocan.
- Tomar la decisión final cuando hay desacuerdo.

Lo que **no** hace habitualmente:

- Escribir código línea por línea.
- Volver a discutir decisiones ya fijadas en los briefs.
- Aprobar PRs sin que la retro de la lane esté escrita.

4.2 Integrador (agente de IA, sesión interactiva)

Responsabilidades:

- Mantener el contexto compartido del proyecto, leyendo el estado del repositorio (*issues*, PRs, retros previas, auditorías).
- Escribir briefs detallados antes de lanzar cada lane.
- Lanzar a los agentes lane en sesiones `tmux` separadas, cada una con su *worktree*.
- Monitorear el progreso usando `tmux capture-pane` y la programación de *wakeups*.
- Reportar los hallazgos críticos al humano.

Lo que **no** hace:

- Tocar el código de la lane directamente.
- Tomar decisiones irreversibles sin autorización del humano.
- Limpiar *worktrees* o sesiones sin autorización explícita.

4.3 Lane (agente de IA, sesión aislada, modo autónomo)

El agente lane corre en **modo autónomo de principio a fin**. En el caso de Claude, esto corresponde a operar con *auto mode on*. El agente no pide confirmación interactiva entre pasos del brief, ejecuta toda la cadena (explorar, diagnosticar, implementar, validar, escribir la retro, hacer *commit*, *push* y abrir el PR) y solo se detiene en los puntos de *STOP-and-report* declarados explícitamente en el brief, o cuando una acción irreversible requiere autorización (ver §9).

La autonomía es lo que hace viable la ejecución paralela. Un humano no puede supervisar N lanes a la vez. El brief reemplaza a la supervisión continua. Por eso su calidad es el techo de la calidad de la lane.

Responsabilidades:

- Recibir un brief al arrancar y leerlo completo antes de actuar.
- Trabajar en su *worktree* aislado, sobre su propia rama.
- Ejecutar el flujo *explorar, diagnosticar, implementar, validar, escribir la retro, hacer commit, push y abrir el PR* sin pausas para pedir confirmación.
- Habilitar la integración automática cuando se espera que el CI termine en verde.
- Hacer ***STOP-and-report*** ante ambigüedades de diseño que excedan su brief.

Lo que **no** hace:

- Improvisar decisiones de alcance.
- Tocar archivos fuera del alcance declarado en el brief.
- Limpiar su propio *worktree* o sesión.

---

5. Anatomía de una lane

5.1 Estructura de archivos

Cada lane vive en tres ubicaciones:

```
/tmp/wt-claude-prompt-<issue>-<descr>.txt   # el brief
/tmp/launch-<issue>.sh                      # script que invoca al agente
~/work/<proyecto>.<issue>-<descr>/          # el worktree con su rama
```

Una sesión `tmux` con nombre `<issue>-<descr>` ejecuta el agente.

5.2 Lanzamiento

```
chmod +x /tmp/launch-NNN.sh
tmux new-session -d -s issue-NNN-descr \
    "wt switch --create issue-NNN-descr -x /tmp/launch-NNN.sh"
```

`wt switch --create` crea el *worktree* junto con la rama, entra al directorio y ejecuta el script. Este invoca `exec claude "$(cat brief)"` y arranca al agente recibiendo el brief como entrada inicial.

5.3 Monitoreo

```
tmux list-sessions
tmux capture-pane -t <session-name> -p | tail -30
```

Si hace falta corregir la trayectoria, se usa `tmux send-keys` para preservar el contexto que el agente ya tiene. **Volver a redactar el brief desde cero pierde el aprendizaje acumulado en la sesión.**

5.4 Limpieza posterior a la integración

Solo después de que el PR de la lane se integró:

```
tmux kill-session -t issue-NNN-descr
git worktree remove ~/work/<proyecto>.issue-NNN-descr
git branch -D issue-NNN-descr
git pull --ff-only origin main
```

Regla fijada: la limpieza necesita autorización explícita del arquitecto. Una limpieza prematura ya ha destruido contexto útil en la práctica.

---

6. El brief como contrato

El brief es la diferencia entre un agente que entrega un PR listo para revisión y uno que pierde el tiempo. Es producto del *briefing* entre arquitecto, integrador y, en su forma final al lanzarse la lane, el agente que la ejecuta. Es una puesta en común explícita sobre qué se va a hacer, con qué decisiones ya cerradas y bajo qué restricciones. La estructura canónica es la siguiente:

6.1 Objetivo

Una a tres frases. Qué se entrega y por qué importa ahora. Idealmente cita el número de *issue*.

6.2 Contexto

- Repositorio y rama (siempre `issue-NNN-descr` desde el `main` actual).
- Verificación esperada (por ejemplo, `git log --oneline -5 origin/main`).
- Reglas fijadas del proyecto (`CLAUDE.md`, archivos que no se tocan).
- Principios arquitectónicos y de negocio aplicables (ver §3.2).
- Lanes paralelas en ejecución, para anticipar conflictos.

6.3 Lectura previa

Lista de archivos que el agente debe leer antes de escribir código:

- El item del *backlog* (*issue* o tarjeta Kanban) completo.
- La auditoría o el documento de diseño relevante, cuando exista.
- Retros de lanes anteriores que comparten contexto.
- Los archivos específicos del código que la lane va a tocar.

6.4 Decisiones fijadas

Decisiones ya tomadas que el agente **no debe discutir de nuevo**. Esta sección es crítica. La IA tiende a "mejorar" lo que ya estaba decidido. Ejemplos típicos:

- *"No arreglar el issue #219 en esta lane."*
- *"No tocar `CHANGELOG.md` ni `VERSION`. Los regenera el incremento al lanzar la versión."*
- *"Cambia lo mínimo para que pase."*
- *"Diagnosticar antes de implementar. El primer paso es entender la falla."*

6.5 Alcance de la lane (dentro y fuera)

Lista explícita de qué SÍ y qué NO hacer.

Lo que NO se hace es tan importante como lo que SÍ. El modelo está entrenado para ser útil. Si no se le dice que pare, sigue. Casos típicos:

- No tocar otros módulos.
- No tocar archivos de versionamiento (`CHANGELOG`, `VERSION`).
- No abrir *issues* nuevos para vacíos adyacentes (mantener el alcance separado).
- *STOP-and-report* ante las condiciones X.

6.6 Salidas requeridas

- Archivos modificados que se esperan.
- Mensaje de *commit* en el formato del proyecto (típicamente *Conventional Commits*).
- Cuerpo del PR con lo que debe incluir.
- Comandos de cierre: `gh pr create`, `gh pr merge --auto`.

6.7 Recordatorios de disciplina

- Nivel de pruebas requerido antes de hacer *push*.
- Árbol de trabajo limpio en cada *commit*.
- Integración automática habilitada cuando corresponde.

6.8 Instrumentación (obligatoria)

- Registro de las compilaciones en formato TSV.
- Documento de retro en `docs/lane-experience-<lane>.md` con secciones fijas (ver §7.2).

7. Captura empírica

Los métodos formales del espectro actual (SDD, SPDD, BMAD) producen artefactos. ELP **además mide el proceso**. Sin esto, el flujo se degrada en pocas semanas.

7.1 Auditorías de fase 0

Cuando un hito abarca varias zonas o asume una hipótesis no validada, se ejecuta una lane **puramente documental** que valida la premisa con datos antes de escribir código.

Estructura de la auditoría (`docs/<hito>-phase0-audit.md`):

1. Inventario empírico (búsquedas, conteos y rastreos sobre el código actual).
2. Cuantificación de la superficie afectada (referencias por archivo, tipos de cambio).
3. Repaso de opciones (A, B, C, D) con costo, riesgo y estado resultante de cada una.
4. Estrategia de control de calidad (cómo se mantiene verde la compilación en cada PR).
5. Inventario de riesgos.
6. Veredicto y recomendación de orden de fases.

Resultado típico documentado en la práctica: algunas auditorías descartan la propuesta entera con datos empíricos, lo que ahorra semanas de trabajo en un hito equivocado.

7.2 Retros de lane

Cada lane escribe `docs/lane-experience-<lane>.md` **antes** del PR. **Sin retro, no hay PR.** Estructura fija:

- **Métricas objetivas.** Marcas de tiempo de inicio y fin, conteos de invocaciones a *build* y *test*.
- **Inventario de cambios.** Archivos modificados, líneas agregadas y borradas, sitios migrados.
- **Errores de compilación encontrados.** Por clase de error, dónde aparecieron, el arreglo aplicado y los intentos fallidos.
- **Puntos de fricción.** Respuestas a las preguntas específicas que el brief pidió responder.
- **Ambigüedades o elecciones interpretativas.** Donde el agente tuvo que decidir sin que el brief lo guiara.
- **Resumen subjetivo.** Confianza, lo más difícil, lo más fácil, en qué ayudó o estorbó la herramienta.
- **Limitaciones del reporte.**

Las retros sirven a cuatro propósitos:

1. Capturan información mientras el contexto está fresco.
2. Son entrada para la siguiente lane (lecciones aprendidas).
3. Son conjunto de datos para evaluar honestamente la *autoría real del LLM*, es decir, qué porción del trabajo el agente puede completar realmente.
4. Documentan vacíos que se transforman en nuevos *issues*.

7.3 Mediciones (TSV)

Antes de la primera invocación a `make` o equivalente, el brief indica:

```
export LANE="issue-NNN-descr"
date -Iseconds > /tmp/lane-${LANE}-start.txt
```

```
echo -e "timestamp\tcmd\toutcome\telapsed_s" \
    > /tmp/lane-${LANE}-builds.tsv
```

Cada `make` se registra como una línea TSV. Al cerrar la lane, ese contenido se agrega a la retro.

El TSV captura:

- El tiempo de reloj real de la lane.
- Cuántos reintentos necesitó cada nivel de pruebas.
- El tiempo dedicado a depurar frente al de implementación pura.

Es el conjunto de datos que permite evaluar honestamente cuánto cuesta cada lane y comparar lanes similares.

7.4 Objetivos de honestidad

Documentos fijados (`docs/<feature>-honesty-targets.md`) que registran dónde el comportamiento del sistema es **simulado** y dónde es **real**. Por ejemplo:

> *"0 errores en la autocompilación. Aquí está la razón: el tipador hace pre-incref antes de la verificación."*

Esto no infla la métrica. Reporta lo medido, no lo proyectado. La cultura es verificable contra el TSV y las retros.

7.5 Hallazgos como nuevos items del backlog

Cuando un agente identifica un vacío fuera del alcance, el flujo es:

1. Lo documenta en la sección de *puntos de fricción* de la retro.
2. **NO lo arregla.** *STOP-and-report*.
3. El integrador decide si crear un nuevo item en el Kanban (*issue* o tarjeta) ahora.
4. Si se crea, el cuerpo del nuevo item cita la retro como fuente.

Cada vacío nuevo es **su propia lane**. Esto evita que una lane se convierta en una cadena infinita de trabajo accesorio.

---

8. Patrones operativos

8.1 Lanes paralelas y secuenciales

Paralelas cuando:

- Tocan zonas distintas del sistema.
- No comparten archivos consumidores.
- Sus salidas son ortogonales.

Secuenciales cuando:

- Tocan los mismos archivos consumidores.
- Una establece un patrón que la siguiente replica.
- La siguiente necesita la **retro** de la anterior.

Criterio operativo: ¿se pisan en el área de trabajo, o en la lección aprendida?

8.2 STOP-and-report

El brief siempre incluye de forma explícita una instrucción del estilo *"si encuentras X, detente y reporta"*. Casos típicos:

- Un error adyacente al alcance.
- Una decisión de diseño que el brief no cubre.
- La necesidad de tocar más zonas que las anticipadas.

El agente **no improvisa decisiones de alcance**. Reporta y espera.

8.3 Programación de wakeups

Para no exigir consultas constantes del humano y no dejar lanes huérfanas, se programa al integrador para despertar en intervalos definidos.

Reglas operativas:

- El retardo típico va entre 60 y 3600 segundos.
- Saltar el rango de 60 a 270 segundos si no hay urgencia. La caché del prompt dura 5 minutos. Esperas más cortas la pierden.
- El prompt del *wakeup* debe ser independiente. Cuando se dispara, tiene que poder interpretarse sin contexto previo.

8.4 Integración automática con CI en verde

Cuando la protección de rama exige ciertas verificaciones en verde, la lane habilita la integración automática en cada PR. Cuando las verificaciones pasan, la integración se dispara sola.

Excepciones aprendidas:

- Los PRs puramente documentales pueden quedar bloqueados si `paths-ignore` salta el flujo de trabajo y la verificación nunca se genera. La solución es un flujo de trabajo que reporta éxito sin correr nada cuando corresponde saltarlo.
- Las verificaciones secundarias no requeridas pueden estar en rojo y el PR igual se integra. La lección es correr todos los controles de calidad **localmente** antes del *push*. No confiar solo en el CI.

---

9. Reglas absolutas

Reglas no negociables que aplican a todos los actores del flujo:

1. **No tocar archivos de versionamiento** (`CHANGELOG`, `VERSION`). Los regenera el incremento al lanzar la versión.
2. **No hacer push directo a `main`**. Siempre PR e integración automática.
3. **Sin autorización explícita, no `worktree remove` ni `branch -D`.**
4. **No comprometer el trabajo del usuario.** Investigar antes de borrar.
5. **Retros obligatorias** por cada lane.
6. **Auditorías de fase 0** para hitos grandes (más de un PR, hipótesis no validada, varias zonas afectadas).
7. **Honestidad sobre optimismo.** Reportar lo medido, no lo proyectado.
8. **Briefs detallados.** Un brief escueto produce un agente perdido.

9. **Los principios fijados no se discuten dentro de una lane** (§3.2). Cambiar un principio es una decisión consciente del arquitecto.

---

10. Anti-patrones

Patrones documentados como problemáticos que ELP evita de forma explícita:

- **Archivos de documentación prematuros** (TODOs, plan.md, decision.md). Los *issues* y las retros cubren lo persistente. El resto vive en la conversación.
- **Re-arreglar un error fuera de alcance.** Abrir un nuevo *issue*, no extender la lane.
- **Migración con alias supervivientes no documentados.** Cada alias vivo necesita una causa fijada en el cuerpo del PR.
- **Integración automática sin verificar los controles localmente.** Las verificaciones secundarias pueden mentir.
- **Briefs escuetos.** *"Implementa X"* sin contexto produce un agente perdido.

---

11. Cuándo aplicar y cuándo no

11.1 ELP encaja cuando

- El proyecto va a durar meses, no días.
- Se cumplen las **precondiciones de §3**: visión y diseño general escritos, principios arquitectónicos fijados, restricciones temporales claras, y un Kanban (o equivalente) gestionado.
- Hay alcance para varios PRs en paralelo, sobre zonas ortogonales del sistema.
- Hay valor en conservar el aprendizaje entre lanes.
- El arquitecto puede dedicar tiempo a escribir briefs.

El método se probó en proyectos *greenfield*. Su extensión a proyectos con *backlog* preexistente requiere primero explicitar lo implícito (principios, convenciones, contratos no escritos) para que los briefs los puedan citar (§3.4).

11.2 ELP NO encaja cuando

- Es un prototipo desechable. La sobrecarga de briefs y retros no se amortiza.
- Es un arreglo urgente en producción. La cadencia de PRs en paralelo no aplica.
- El equipo no tiene pruebas ni CI maduros. ELP **amplifica** lo que ya existe. Sin una tubería sólida, amplifica el caos.
- El arquitecto no puede dedicar entre 10 y 20 minutos por brief. La calidad del brief es el techo de la calidad de la lane.
- No existe una visión de producto ni un diseño general previo. ELP no inventa el *qué construir*, lo orquesta.

---

12. Comparación con métodos cercanos

12.1 ELP frente a BMAD-METHOD

BMAD divide por **especialidad funcional**, con 12 o más agentes especialistas (jefe de producto, arquitecto, desarrollador, UX, QA, etc.) que el humano va activando a lo largo de fases agile. ELP

divide por **unidad paralelizable**, con lanes generalistas y alcance acotado, cada una en su propia sesión.

Dimensión	BMAD	ELP
División del trabajo	por especialidad	por unidad paralelizable
Topología	secuencial por fases, con colaboración multi-agente dentro de la sesión (*party mode*)	varias lanes paralelas e independientes, en sesiones aisladas
Artefacto central	PRD, documento de arquitectura, historias de usuario	brief y retro por lane
Fuente del *qué construir*	PRD generado por el agente jefe de producto	items del *backlog* gestionado en Kanban
Captura del proceso	parcial	persistente (auditorías, retros y TSV)
Aislamiento de cambios	una rama, dentro del IDE	un *worktree* y una sesión `tmux` por lane

12.2 ELP frente a SPDD

SPDD trata al prompt como **artefacto vivo**, versionado en sincronía con el código, con plantillas como el *REASONS Canvas* para forzar claridad. ELP trata al **brief** como contrato de un solo uso, pero acumula aprendizaje en las **retros**, que sí se versionan. Son ortogonales y se pueden combinar: nada impide usar el *REASONS Canvas* como estructura del brief de una lane ELP.

12.3 ELP frente a SDD

SDD trata a la **especificación** como artefacto persistente y fuente de verdad: el código se considera un derivado regenerable desde la spec, y los cambios de requisitos son re-generaciones sistemáticas. ELP invierte la prioridad: el código integrado es el artefacto persistente, el brief es un contrato táctico de un solo uso, y la **retro** acumula el aprendizaje que la spec descartable de SDD no captura. Las dos visiones son complementarias: una spec viva al estilo SDD puede coexistir con lanes ELP que la implementan en paralelo.

12.4 ELP frente a Pantser

El término *Pantser* aplicado al desarrollo de software fue articulado por Eduardo Díaz en *Mapas o Brújulas* (lnds.net, 2025), tomado prestado de la literatura. El *Pantser* es quien escribe **by the seat of his pants**, sin un mapa rígido, ajustando la dirección con cada paso. Su contraparte clásica es el *Plotter*, que planifica meticulosamente antes de ejecutar.

El *Pantser* itera empíricamente con revisión humana disciplinada, pero el aprendizaje **muere con la conversación**. ELP captura ese aprendizaje en formatos persistentes que sobreviven al cierre de la sesión. La diferencia central es la **persistencia del aprendizaje**, no la presencia del empirismo.

---

13. Métricas

Métricas que ELP recomienda capturar a nivel de proyecto:

- **PRs integrados** por día y por semana.
- **Lanes paralelas en su pico** de concurrencia simultánea.
- **Porcentaje de lanes que terminan en *STOP-and-report* útil**, como indicador de la calidad del brief.
- **Tiempo de reloj por lane** frente al estimado, desde el TSV.
- **Reintentos por lane**, separando depuración real e implementación inicial.
- **Auditorías de fase 0 ejecutadas** y **porcentaje que descartó el hito**.
- ***Issues* abiertos como seguimiento** desde las retros, como señal de salud del flujo.

---

14. Caso de estudio: compilador para un lenguaje funcional con efectos algebraicos

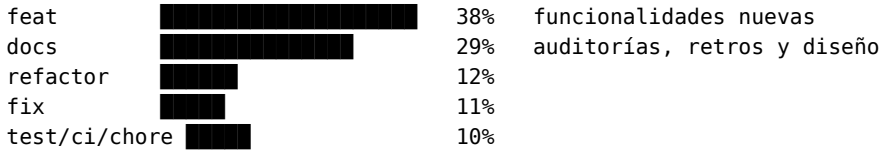
ELP se formalizó a partir de la práctica acumulada en un proyecto concreto: la construcción desde cero de un **lenguaje de programación funcional autocompilable con efectos algebraicos** y *backend* LLVM, ejecutado entre el **20 de abril y el 4 de mayo de 2026**, en 14 días corridos. Este proyecto es *greenfield* y partió con visión de producto, diseño general, ejemplos de sintaxis objetivo y principios arquitectónicos fijados desde el día cero (§3).

14.1 Cifras crudas

Indicador	Valor
Tiempo total	**14 días corridos**
Commits	**868**
Pull requests integrados	**144**
Issues cerrados	**69**
Lanzamientos	v0.1.0 a **v0.31.0**
Pico de *commits* en un día	**135**
Auditorías de fase 0 ejecutadas	3
Retros de lane producidas	más de 25
Pico de lanes paralelas	3 simultáneas

Las cifras no son percepción. Salen del `git log`, del `gh pr list` y del TSV de cada lane. Cada PR cita la retro que lo precede.

14.2 Distribución de los commits



29 % de documentación es alto para un compilador. Es justo el conjunto de datos empíricos que ELP recomienda: auditorías de fase 0, retros, documentos de diseño por funcionalidad, objetivos de honestidad. **11 % de arreglos es bajo**. Las funcionalidades llegaron limpias gracias a los controles

de calidad (autocompilación byte a byte desde el día 1, nivel 1 de pruebas obligatorio antes del *push*, retro obligatoria antes del PR).

> Pagar por adelantado el 29 % de documentación es lo que mantiene el 11 % de arreglos.

14.3 Lo que se construyó

Un lenguaje funcional autocompilable con propiedades poco frecuentes en una sola implementación:

- **Compilador en tres etapas.** Etapa 0 en C (unas 10K líneas), etapa 1 como puente (unas 6K), etapa 2 autocompilable (unas 41K líneas).
- **Autocompilación byte a byte** como control de calidad desde el día 1. Cada cambio debe poder recompilarse a sí mismo con bytes idénticos.
- **Sistema de efectos** al estilo Effekt: capacidades, manejadores, inferencia de filas, efectos como `Console`, `Spawn`, `Cancel`, `Actor`, `Mutable`, `Ffi`.
- **Conteo de referencias estilo Perceus** (Koka). Conteo compilado en tiempo de compilación, sin verificador de préstamos, con reutilización en sitio y eliminación de incrementos y decrementos redundantes.
- **Fibras al estilo BEAM.** Montículo privado por fibra, mensajes copiados, planificador cooperativo con `swapcontext`.
- **Biblioteca estándar y herramientas completas:** `kai build/run/test/fmt/repl/lsp/doc/bench/check`.
- **Optimización de llamadas en cola obligatoria.** Es una regla del lenguaje. Las auto-llamadas en cola se reescriben como bucles con `goto` en la emisión a C.

14.4 Comparación con la industria

Proyectos públicos comparables y su tiempo hasta lograr autocompilación funcional:

Proyecto	Tiempo	Equipo
Roc (efectos y funcional)	unos 5 años	investigación
Koka (inventó Perceus)	más de 10 años	académico
Effekt (capacidades)	unos 5 años	PhD
Crystal (LLVM e inferencia)	unos 3 años	equipo principal

Equivalente humano para alcanzar el mismo alcance técnico:

> **De 3 a 4 ingenieros senior expertos en compiladores, durante 12 a 18 meses, equivalente a entre USD 1.0 y 1.5 millones en remuneraciones.**

ELP llegó al mismo punto en **12 a 14 días, con un humano (el arquitecto), un agente integrador y lanes paralelas**.

14.5 Qué aportó el método

Atribución honesta del resultado, sin extrapolar:

1. **Autocompilación byte a byte desde el día 1.** Cualquier cambio que rompa la autocompilación se descubre en la compilación, no en producción. Blindó cada funcionalidad contra regresiones invisibles.
2. **Arranque en tres etapas.** Permite que la etapa 2 use el lenguaje completo sin limitarse al subconjunto que un compilador escrito solo en C podría implementar. La etapa 1 actúa como puente.
3. **Auditorías de fase 0 como cultura.** Tres auditorías ya ahorraron semanas. Una descartó por completo una propuesta de partir el arranque, con datos empíricos. La premisa era falsa. Sin la auditoría, el equipo habría implementado sobre una hipótesis equivocada.
4. **Retros como conjunto de datos, no como documentación.** Las retros que registran errores de compilación, caminos de arreglo fallidos y puntos de fricción alimentan a la siguiente lane y son el dato real para evaluar la *autoría real del LLM*.
5. **Objetivos de honestidad fijados.** Cada funcionalidad documenta dónde su comportamiento es simulado y dónde es real. Por ejemplo, *"0 errores en autocompilación. Aquí está la razón."* La cultura escala porque la honestidad es verificable contra los TSV.
6. ***Backlog* como hoja de ruta.** El *qué construir* funcional vive en el Kanban (en este caso, los *issues* de GitHub como sustituto funcional), separado del *cómo*, que vive en el brief. Esto evita que cada lane se convierta en una cadena de trabajo accesorio.

14.6 Lo que se cerró como negativo

Decisiones que no calzaron, fijadas como *negativos*:

- *Drop specialisation*. Se midió un –1,7 % con `-O2` y un +5,4 % de regresión con `-O0`. La lane se cerró como negativo. El desempaquetado de fase 2 ya había absorbido la sobrecarga.
- *Bootstrap split*. Descartado por auditoría empírica antes de implementarse. La fase 0 ahorró entre 2 y 3 semanas de trabajo equivocado.

Estos negativos no son fallas, son **datos**. Cada uno tiene una retro fijada que evita repetirlos.

14.7 Observación final

El proyecto tiene una propiedad poco frecuente. Es **técnicamente ambicioso y operacionalmente honesto** al mismo tiempo. Las auditorías de fase 0, las retros y los objetivos de honestidad construyen un conjunto de datos que un equipo humano normalmente posterga o pierde. El humano, el arquitecto, no escribió código. Mantuvo el juicio sobre el alcance, las decisiones de arquitectura y los llamados a la prudencia del tipo "esto no me convence". Los agentes ejecutaron las lanes con briefs detallados.

Si la hipótesis de fondo se sostiene (la autoría real del LLM, validable contra el conjunto de datos acumulado), este caso es la primera evidencia empírica documentada de un proyecto técnicamente complejo construido bajo ELP.

---

15. Limitaciones conocidas

- Si el arquitecto está durmiendo y un agente hace *STOP-and-report*, la lane queda **bloqueada** hasta que el humano responda. Una mitigación práctica es incluir en el brief autorizaciones preaprobadas para los casos comunes.
- Si el CI tarda y el *wakeup* dispara antes, se gasta caché del prompt sin avance. Es una compensación consciente.

- Los briefs detallados toman entre 10 y 20 minutos del arquitecto. No siempre vale la pena para lanes triviales, como la edición de un solo archivo.
- El método asume herramientas de versionamiento y CI maduras. En proyectos sin ellas no aplica directamente.
- Cambiar de máquina o reiniciar la sesión del integrador pierde el contexto vivo. Hoy no hay solución estructural.

---

16. Origen y referencias

ELP se formalizó a partir de la práctica acumulada en proyectos de larga duración con varios agentes de IA trabajando en paralelo. La primera articulación pública del método aparece en este documento.

Métodos y referencias relacionadas:

- **Díaz, E.,** *"Mapas o Brújulas"*, lnds.net, 2025-10-31. URL: https://lnds.net/blog/lnds/2025/10/31/mapas-o-brujulas/. Articulación en español del binomio *Plotter* y *Pantser* aplicado al desarrollo de software.
- **BMAD-METHOD** (*Breakthrough Method for Agile AI-Driven Development*), de Brian Madison. Tubería multiagente con roles funcionales. Repositorio: https://github.com/bmad-code-org/BMAD-METHOD. Documentación: https://docs.bmad-method.org/. Masterclass oficial en YouTube: https://www.youtube.com/watch?v=LorEJPrALcg.
- **SDD** (*Spec-Driven Development*). La especificación como fuente de verdad y artefacto persistente desde el cual se regenera el código. Implementación de referencia: GitHub Spec Kit, https://github.com/github/spec-kit. Documentación: https://github.github.com/spec-kit/. Tutorial completo en YouTube: https://www.youtube.com/watch?v=xfyec__ieHA.
- **SPDD** (*Structured Prompt-Driven Development*), de Wei Zhang y Jessie Jie Xia (Thoughtworks, 2026). El prompt como artefacto vivo, con el *REASONS Canvas* como plantilla. Artículo en martinfowler.com: https://martinfowler.com/articles/structured-prompt-driven/. Video explicativo en YouTube: https://www.youtube.com/watch?v=CotFOAGtb64.
- **Saltzer y Schroeder**, *The Protection of Information in Computer Systems*, 1975. Los principios de diseño seguro que aplican igual al agente.
- **Karpathy, A.,** *"Vibe Coding"*, 2024.
- **Krouse, S.,** *"Vibe Code is Legacy Code"*, 2025.
- **Emerson, Lake & Palmer**, *Karn Evil 9: First Impression Part 2*, 1973. Origen del acrónimo retrospectivo ELP.

Apéndice A. Plantilla mínima de brief

```
# Brief de la lane: issue-NNN-<descr>

## Objetivo
[1 a 3 frases. Qué se entrega y por qué.]

## Contexto
- Repositorio: <repo>, rama: issue-NNN-<descr>
- Verificación: git log --oneline -5 origin/main
- Reglas fijadas del proyecto: CLAUDE.md
```

```
- Lanes paralelas en ejecución: [lista]

## Lectura previa
- Issue: gh issue view NNN
- Auditoría: docs/<hito>-phase0-audit.md (si aplica)
- Retros relacionadas: docs/lane-experience-<previo>.md
- Archivos a inspeccionar: <rutas>

## Decisiones fijadas
- [decisión 1]
- [decisión 2]

## Alcance de la lane
SÍ:
- [acción 1]
- [acción 2]

NO:
- [prohibición 1]
- [prohibición 2]

STOP-and-report si:
- [condición 1]

## Salidas requeridas
- Archivos esperados a modificar: <rutas>
- Mensaje de commit: <tipo>(<alcance>): <asunto>
- Cuerpo del PR: [qué incluir]
- Cierre: gh pr create + gh pr merge --auto

## Disciplina
- Nivel 0 y nivel 1 de pruebas en local antes del push.
- Árbol de trabajo limpio en cada commit.

## Instrumentación
- TSV de compilaciones en /tmp/lane-${LANE}-builds.tsv.
- Retro en docs/lane-experience-issue-NNN-<descr>.md (secciones obligatorias).
```

Apéndice B. Plantilla mínima de retro de lane

```
# Retro de la lane: issue-NNN-<descr>

## Métricas objetivas
- Inicio: <marca de tiempo ISO>
- Fin: <marca de tiempo ISO>
- Duración total: <duración>
- Invocaciones de compilación: <conteo por nivel>

## Inventario de cambios
- Archivos modificados: <lista>
- Líneas agregadas y borradas: +X / -Y
- Sitios tocados: <conteo>

## Errores de compilación encontrados
```

```
- [Clase de error]: dónde, arreglo aplicado, intentos.

## Puntos de fricción
- [Pregunta del brief]: respuesta basada en lo observado.

## Ambigüedades o elecciones interpretativas
- [Decisión sin especificación explícita]: justificación.

## Resumen subjetivo
- Confianza: alta, media o baja.
- Lo más difícil: ...
- Lo más fácil: ...
- En qué ayudó o estorbó el compilador o la herramienta: ...

## Limitaciones del reporte
- [Qué no cubre este reporte.]
```